Experiment No : 01

DATE

Group A

* Title

Write a python program to manage the borrowing records of books in a library implement the following functionalities.

1) Compute the average number of books borrowed by all library members.

2) Find the book with the highest and lowest number of borrowing in the library.

3) Count the number of member who have not borrowed any books (denoted by a borrow count of 0).

4) Display the most frequently book (i.e. the mode of borrow counts).

After performing determine the time and space complexity of each operation.

* Objectives :-

1) To provide hands on experience with basic and advanced data structures.

2) To apply data structure for real world problem solving.

* Learning Objectives :-

To understand importance of data structure in context of writing efficient programs.

FOR EDUCATIONAL USE

* Learning Outcome :-

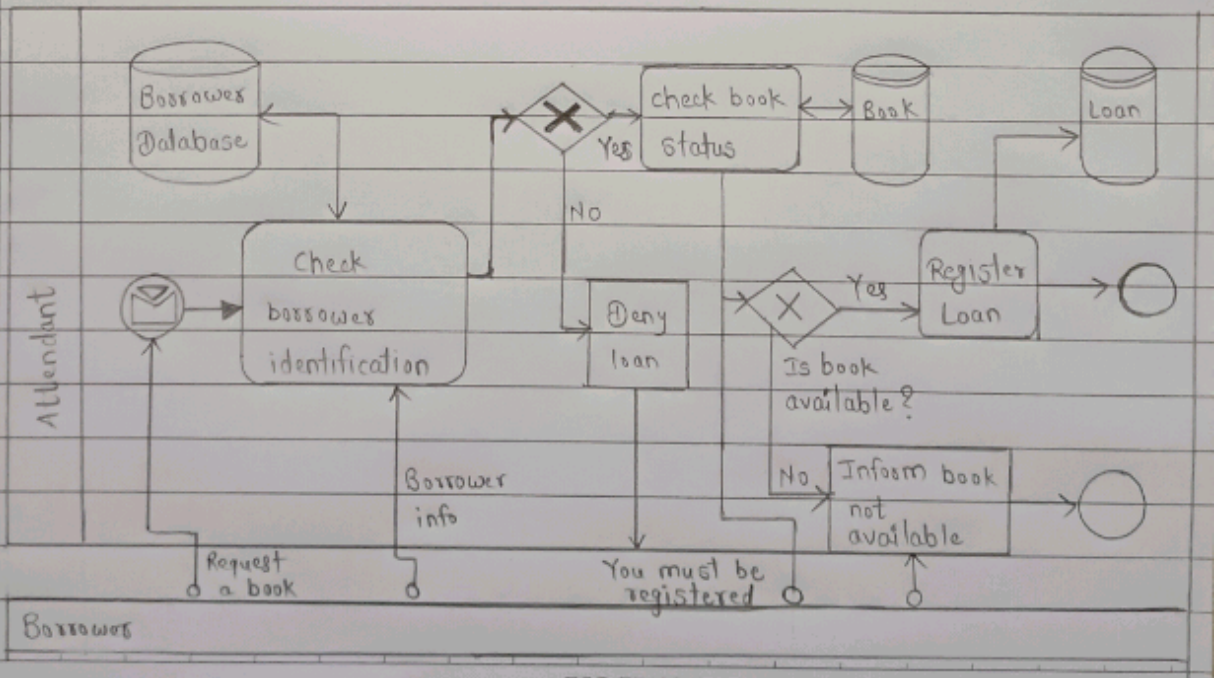
1) Analyze basic searching and sorting algorithm to solve problem and evaluate their efficiency in different scenerious.

2) Understand and analyze various type of data structure and algorithms.

* Theory :-

A library management system using python GUI typically involves creating an application that manages book records, member information and borrowing processes can build an interactive system that allows user to add update and search for book efficiently.

The architecture of a library management system (LMS) using python typically follows a layered structure that enhances modularity and maintainability.



FOR EDUCATIONAL USE

The components and fundionalities

1) Book Management :-

- Adding Books :-
  Allows librarians to add new books to the system, including details like title, author, ISBN and quantity.
- Viewing Books :-
  Displays a list of all books in the library, including their details.
- Updating Books :-
  Enables modification of book information (e.g. quantity, location)
- Removing Books :-
  Allows for deleting books from the system.

2) Member Management :-

- Adding Members :-
  Allows registration of new library members with their details.
- Viewing members :-
  Displays a list of all registered members.
- Updating members :-
  Allows modification of member information
- Deleting members :-
  Removes members from the system.

3) Borrowing / Returning :-

- Borrowing Books :-
  Links a book to a member's ID and records the borrowing date and due date.

FOR EDUCATIONAL USE

- Returning Books :-
  Updates the book's availability and removes the borrowing record.
- Overdue Books :-
  Tracks books with a past due date and may include a mechanism for lines.

* Space Complexity

- Definition :-
  Space complexity measures the amount of memory or storage space an algorithm needs to complete it's execution relative to the input size (n).

- Components :-
  It considers both the fixed part (space for code, constants, simple variables) and the variable part (space for dynamic data structures, recursion, stack, etc.)

- Notation :-
  Like time complexity, space complexity is also commonly expressed using Big O notations.

- Examples:
  O(1) (constant space) :- Algorithms using a fixed amount of memory regardless of input size, like swapping two numbers or an in-place sort like bubble sort.

- O(n) (linear space) :- Algorithms where memory usage grows linearly with input such as creating an array of 'n' elements.

FOR EDUCATIONAL USE

- O (log n) (Logarithmic Time) :- Operations where the time taken increases logarithmically with input size, like binary search on a sorted array.

- O(n) (Linear Time) :- Operations where the time taken increases linearly with input size, like iterating through an array or linked list to find an element.

- O (n log n) (Log - Linear / Linearithmic Time) :- Common in efficient sorting algorithms like merge sort, where the time grows proportionality to the input size multiplied by the logarithm of the input size.

- O (n^2) (Quadratic Time) :- Operations involving nested loops, where the time taken grows proportionally to the input size multiplied by the logarithm of the input size, proportionality to the square of the input size, like simple sorting algorithms such as bubble sort, selection sort and insertion sort.

Conclusion :-

By leveraging python's capabilities and object-oriented programming principles, libraries can create a system that is efficient, user-friendly and adaptable to their specific needs. The potential for growth and integration of new technologies ensure that the system can evolve alongside the changing landscape of libraries and information management.

FOR EDUCATIONAL USE

or using data structures like linked lists or stacks.

- $O(n^2)$ (Quadratic Space) :- Algorithms requiring memory proportional to the square of input size, often seen in algorithms that create two-dimensional arrays based on the input size.

* Time Complexity

- Definition :-

Time complexity measures the amount of time an algorithm takes to run to completion as a function of the input size (n).

- Focus :-

It focuses on the number of operations or computational steps the algorithm performs, rather than the actual time taken, which can be influenced by factors like the programming language, hardware and operating system.

- Notation :

Time complexity is typically expressed using asymptotic notations like Big O, Big Theta (Θ) and Big Omega (Ω) with Big O (worst-case scenario) being the most commonly used.

- Examples :-
- O(1) (constant time) :- Operations that take the same amount of the time regardless of input size, like accessing an element in an array by its index.

FOR EDUCATIONAL USE